

第六章 全局寄存器分配

冯洋

南京大学计算机学院
fengyang@nju.edu.cn

全局寄存器分配

- 局部寄存器分配没有解决变量重用问题
- 全局寄存器分配通常采用图染色算法

全局寄存器分配

- 局部寄存器分配没有解决变量重用问题
- 全局寄存器分配通常采用图染色算法
 - 需要构建冲突图

全局寄存器分配

- 局部寄存器分配没有解决变量重用问题
- 全局寄存器分配通常采用图染色算法
 - 需要构建冲突图
 - 找到一个最少k个颜色用于这个图的染色

全局寄存器分配

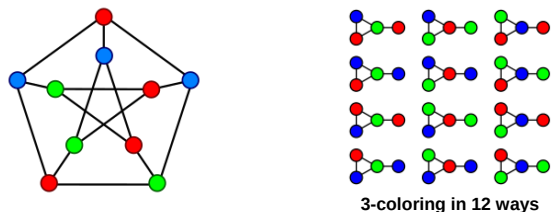
- 局部寄存器分配没有解决变量重用问题
- 全局寄存器分配通常采用图染色算法
 - 需要构建冲突图
 - 找到一个最少k个颜色用于这个图的染色
 - NP-Complete 问题，因此需要一些比较好的启发用以改进

全局寄存器分配

- 点着色：在同一条边连接的两个点上染上不同的颜色（非常简单）
- K-Coloring：最多使用K种颜色

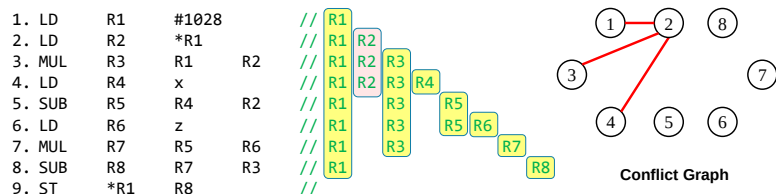
全局寄存器分配

- 点着色：在同一条边连接的两个点上染上不同的颜色（非常简单）
- K-Coloring：最多使用K种颜色



寄存器分配中的K-Coloring 算法

- 我们把上面问题中的结点映射为 **虚拟寄存器**
- 把颜色映射为 **物理寄存器**



寄存器分配中的K-Coloring 算法

- 其中，结点的“度（degree）”，是染色的一个 loose upper bound
 - 即，对于一个“度” < k，那么这个图一定是可以通过k种颜色完成染色的

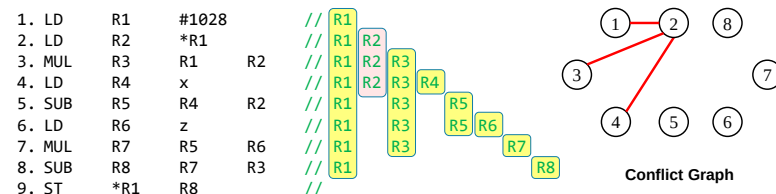
- Chaitin' s 算法 （1982）

寄存器分配中的K-Coloring 算法

- 我们把上面问题中的结点映射为 **虚拟寄存器**
- 把颜色映射为 **物理寄存器**
- 从前面的讨论过的生存期（live）范围构建一个冲突图
- 基于前面的算法进行染色
 - 即实现“同一条边上没有两个虚拟寄存器使用了同一个物理寄存器”
 - 如果发现无法实现，则需要“溢出”

寄存器分配中的K-Coloring 算法

- 我们把上面问题中的结点映射为 **虚拟寄存器**
- 把颜色映射为 **物理寄存器**



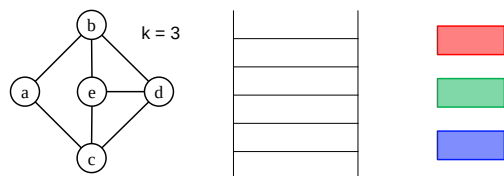
如果使用k种颜色完成这个染色，则说明，我们的N个虚拟寄存器，实际上只需要使用k个物理寄存器

Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个 loose upper bound
 - 即，对于一个“度” < k，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空

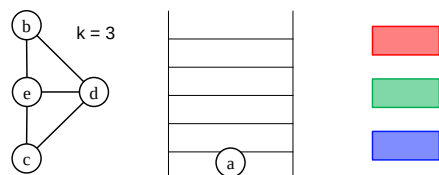
Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个loose upper bound
 - 即，对于一个“度” $< k$ ，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空



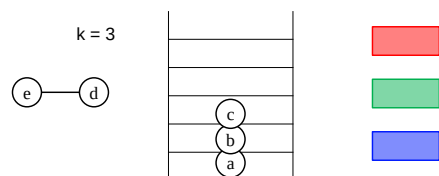
Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个loose upper bound
 - 即，对于一个“度” $< k$ ，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空



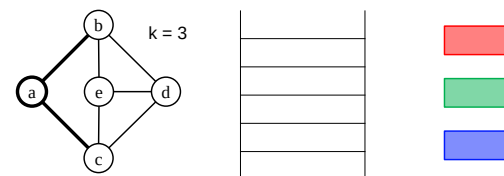
Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个loose upper bound
 - 即，对于一个“度” $< k$ ，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空



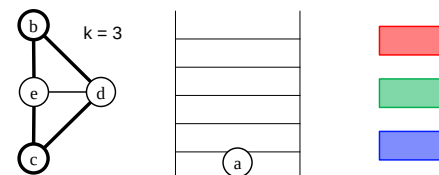
Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个loose upper bound
 - 即，对于一个“度” $< k$ ，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空



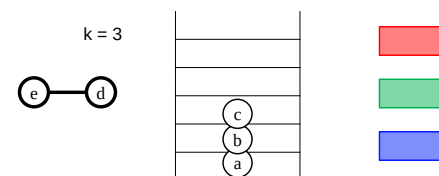
Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个loose upper bound
 - 即，对于一个“度” $< k$ ，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空



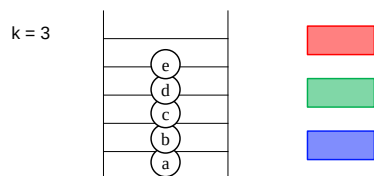
Chaitin' s 算法

- 其中，结点的“度（degree）”，是染色的一个loose upper bound
 - 即，对于一个“度” $< k$ ，那么这个图一定是可以通过k种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中，直到图变空



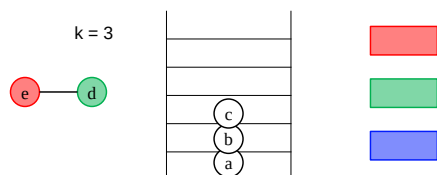
Chaitin' s 算法

- 其中, 结点的“度 (degree)”, 是染色的一个 loose upper bound
 - 即, 对于一个“度” $< k$, 那么这个图一定是可以通过 k 种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中, 直到图变空



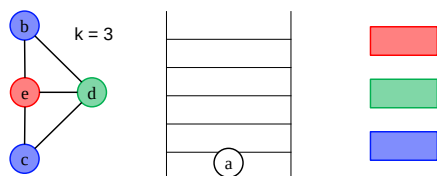
Chaitin' s 算法

- 其中, 结点的“度 (degree)”, 是染色的一个 loose upper bound
 - 即, 对于一个“度” $< k$, 那么这个图一定是可以通过 k 种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中, 直到图变空



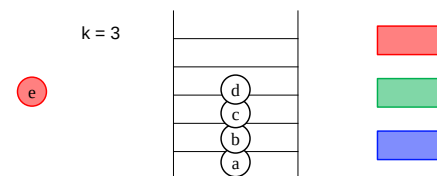
Chaitin' s 算法

- 其中, 结点的“度 (degree)”, 是染色的一个 loose upper bound
 - 即, 对于一个“度” $< k$, 那么这个图一定是可以通过 k 种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中, 直到图变空



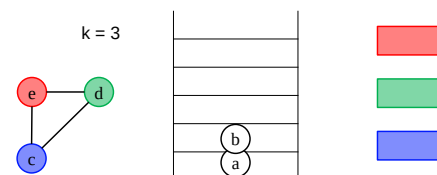
Chaitin' s 算法

- 其中, 结点的“度 (degree)”, 是染色的一个 loose upper bound
 - 即, 对于一个“度” $< k$, 那么这个图一定是可以通过 k 种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中, 直到图变空



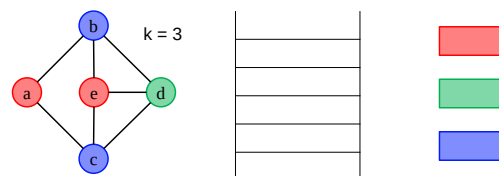
Chaitin' s 算法

- 其中, 结点的“度 (degree)”, 是染色的一个 loose upper bound
 - 即, 对于一个“度” $< k$, 那么这个图一定是可以通过 k 种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中, 直到图变空



Chaitin' s 算法

- 其中, 结点的“度 (degree)”, 是染色的一个 loose upper bound
 - 即, 对于一个“度” $< k$, 那么这个图一定是可以通过 k 种颜色完成染色的
 - 把一些“度”比较高的结点移入栈中, 直到图变空



第六章 代码生成

冯洋

南京大学计算机学院

fengyang@nju.edu.cn